RAPPORT DE PROJET DJANGO

FRAMEWORK PYTHON FLASQUE, TORNADO, FLUTTER, DJANGO

Prof :
LAWSON Lativisena

Rapport de :
HEMOU S. E. Hyacinthe

Sommaire

I. Diagrammes

1. Diagramme de Cas d'utilisation
2. Diagramme de Classe
3. Daigramme de Sequence

II. Test des (06) Scenarios

1. Test 1 : Authentification
2. Test 2 : Création de Tickets
    a. Remplissage du formulaire Flutter et Soumission
    b. Connexion en tant qu'admin et vérification
3. Test 3 : Liste des tickets avec Filtre
    a. Via Flutter
    b. Via l'interface Django
4. Test 4 : Changement de statut
5. Test 5 : Changement de statut
6. Test 6 : Token expire

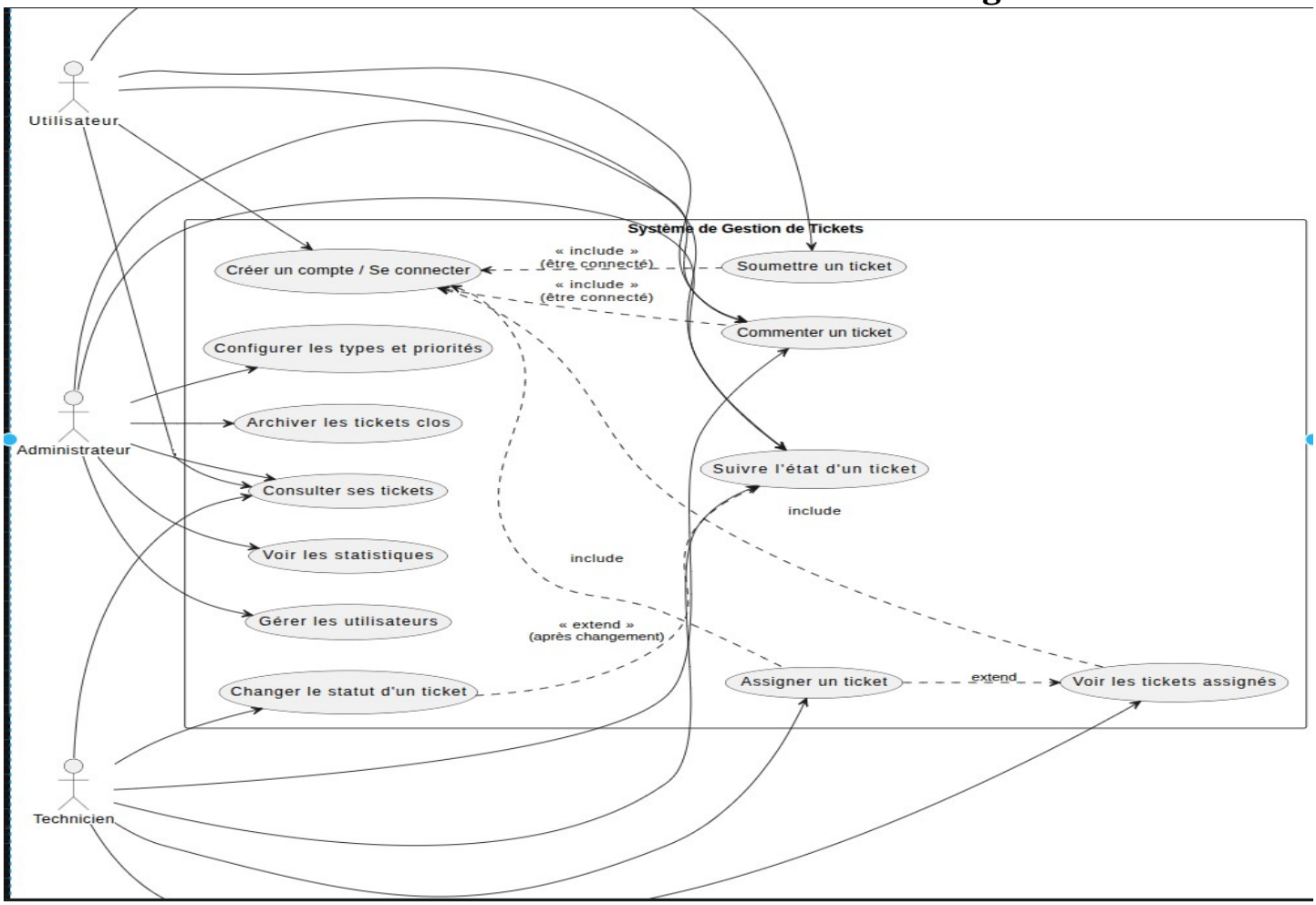
Annex

I. Diagrammes (confer Diagramme.zip)

De peur de perdre la qualité des images (.png) des diagrammes réalisé avec le logiciel draw.io, j'ai decidé de garder le format par defaut (.drawio) qui necessite le logiciel lui même avant d'y accéder.

1. Diagramme de Cas d'Utilisation (User Cas Diag.)

Diagramme de class.



Explications

Acteurs

- Utilisateur (générique).
- Administrateur (hérite d'Utilisateur).
- Technicien (hérite d'Utilisateur).

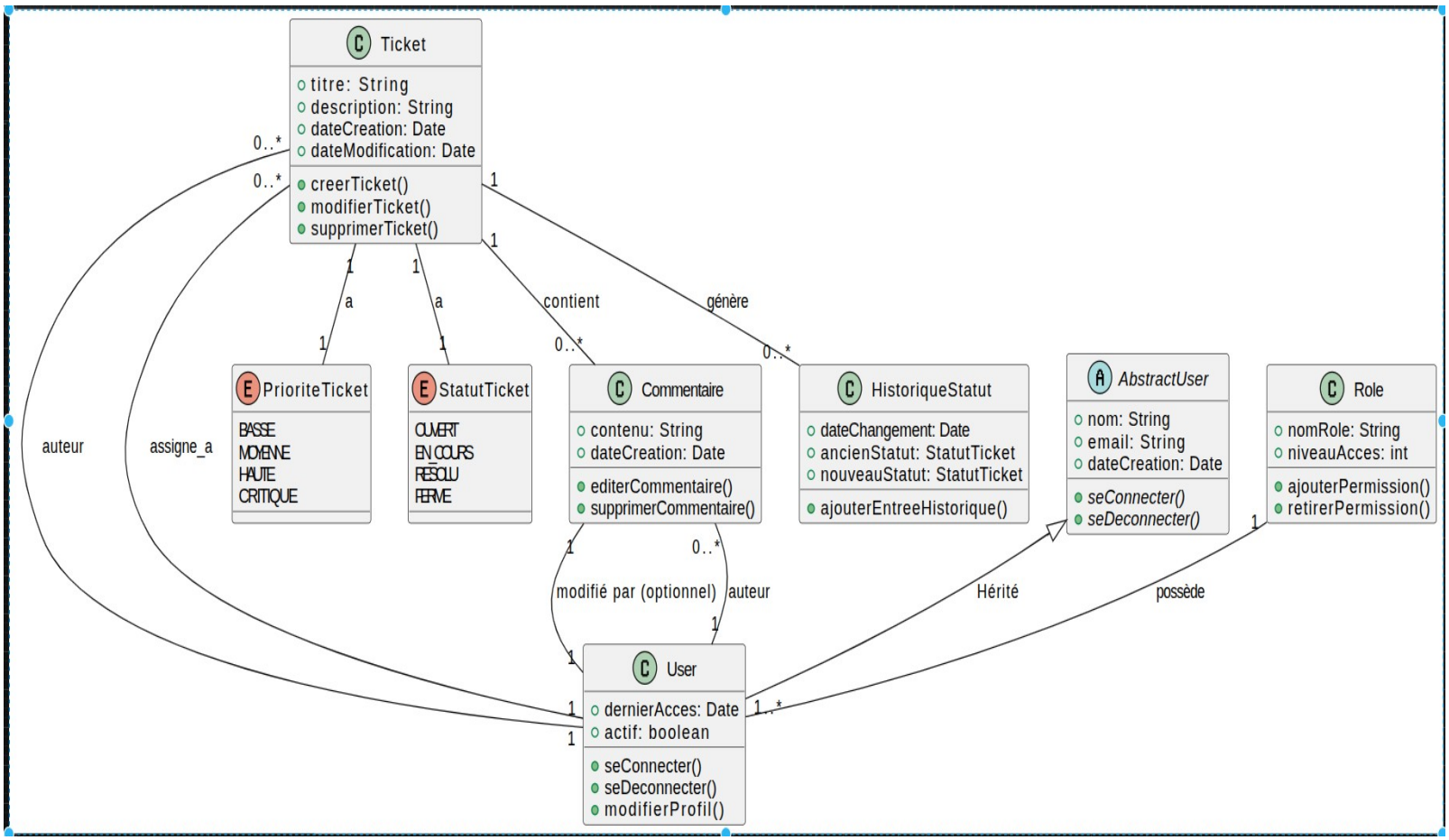
Principaux cas d'utilisation

- Créer un compte / Se connecter.
- Configurer les types et priorités (admin).
- Archiver les tickets clos.
- Suivre l'état d'un ticket.
- Assigner un ticket.
- Voir les tickets assignés.
- Voir les statistiques.
- Voir les statuts.

Relations

- « **include** » : une étape obligatoire (ex. : être connecté).
- « **extend** » : une étape optionnelle après un changement.

2.Diagramme de Classe (Class Diag.)



Explication

Acteurs principaux

- ***AbstractUser*** (classe abstraite) : Elle contient les attributs communs (*nom*, *email*, *dateCreation*) et méthodes (*seConnecter*, *seDeconnecter*).
- ***Auteur*** : C'est une classe liée à l'assignation (*assigne_a*).

Gestion des tickets

- ***Ticket*** : titre, description, dates, méthodes ***CRUD***.
- ***PrioritizeTicket*** : gère la priorité (exemple *BASE_MONE_HUJE_CRITIQUE*).
- ***StatutTicket*** : gère l'état (exemple *QUART_ENCOURS_RESULTURE*).
- ***HistoriqueStatut*** : trace les changements de statut (date, ancien/nouveau...).

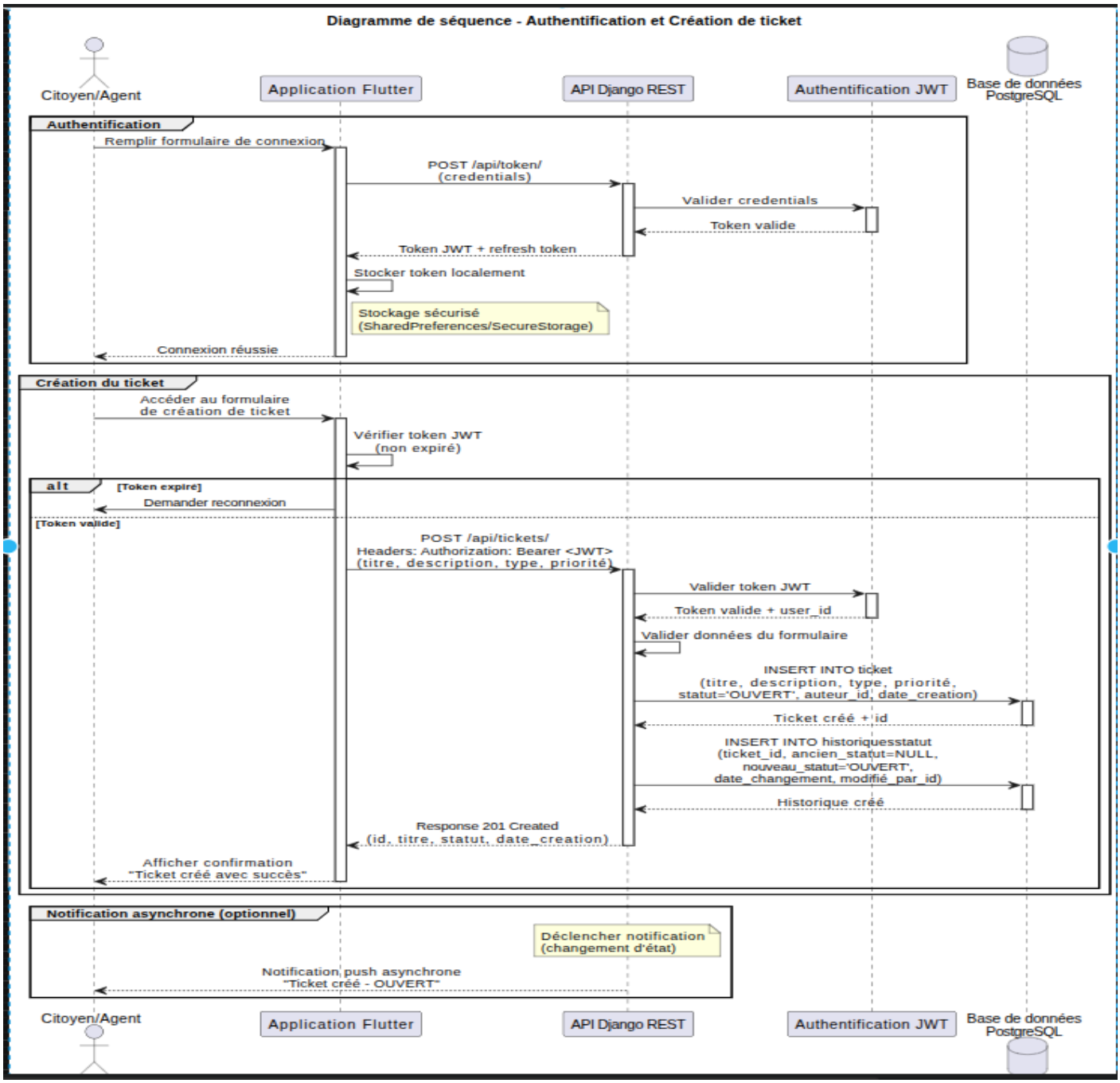
Interactions

- ***Commentaire*** : contenu, date, méthodes d'édition/suppression.
- ***Role*** : définit les permissions (nomRole, niveauAcces).

Relations

- ***Héritage*** : certaines classes héritent d'AbstractUser.
- ***Possède*** : un Ticket peut avoir des Commentaires, un Statut, une Priorité.

3. Diagramme de Sequence (Sequence Diag)



Explication

Authentification

- L'utilisateur remplit le formulaire de connexion.
- L'API valide les identifiants et retourne JWT + refresh token.
- L'application stocke les tokens localement (SharedPreferences/SecureStorage).

Création d’un ticket

- L’utilisateur accède au formulaire de création.
- L’application envoie une requête POST /api/tickets/ avec le JWT dans le header.
- L’API valide le token et les données du formulaire.
- Insertion du ticket en base (statut = ‘OUVERT’).
- Insertion dans historiquesstatut (traçabilité).
- L’application affiche une confirmation.

Rôle des composants

- Flutter : interface utilisateur + stockage local du token.
- Django REST + JWT : sécurité, validation, logique métier.
- PostgreSQL : persistance des tickets et historiques.

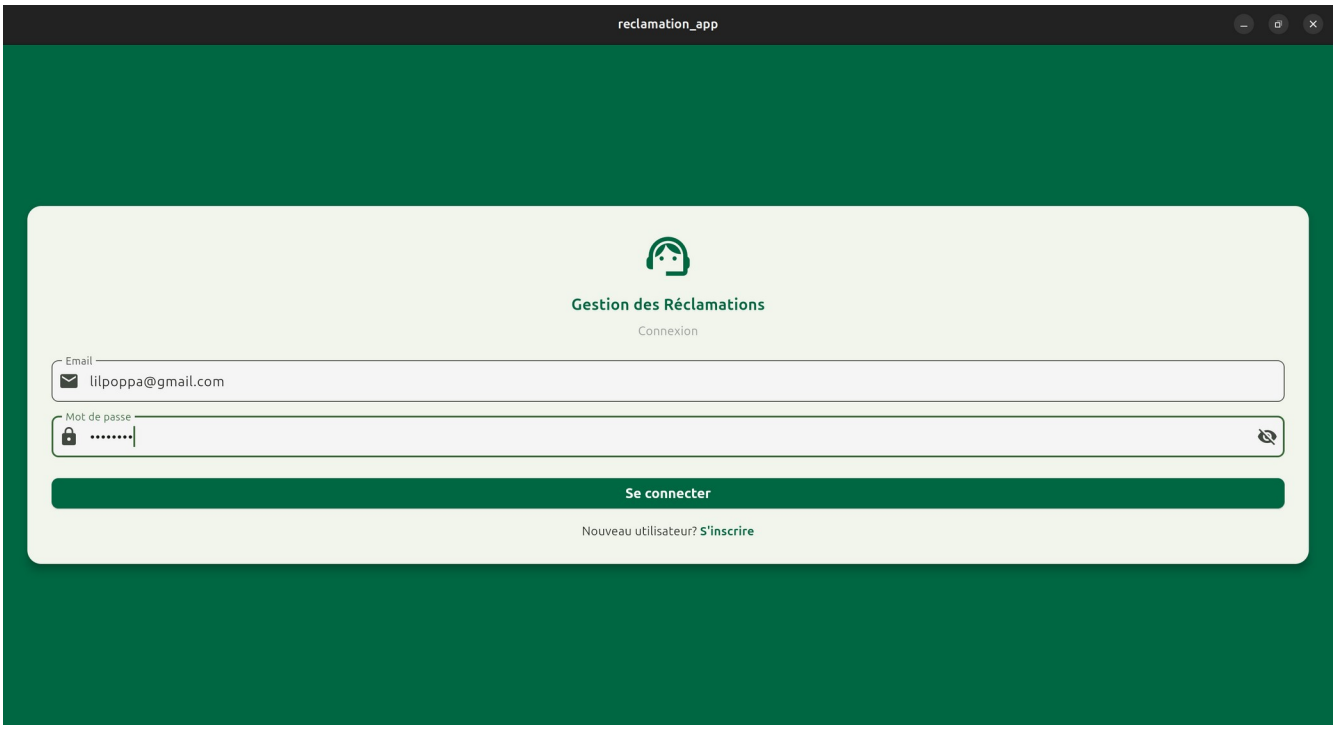
II. Test des (06) Scenarios

1. Test 1 : Authentification

Sujet :
Authentification : Un citoyen se connecte depuis Flutter avec ses identifiants Django. L'access token est stocke dans SharedPreferences. Verifier que les requetes suivantes portent le bon header Authorization.

CONNEXION

page de connexion de flutter

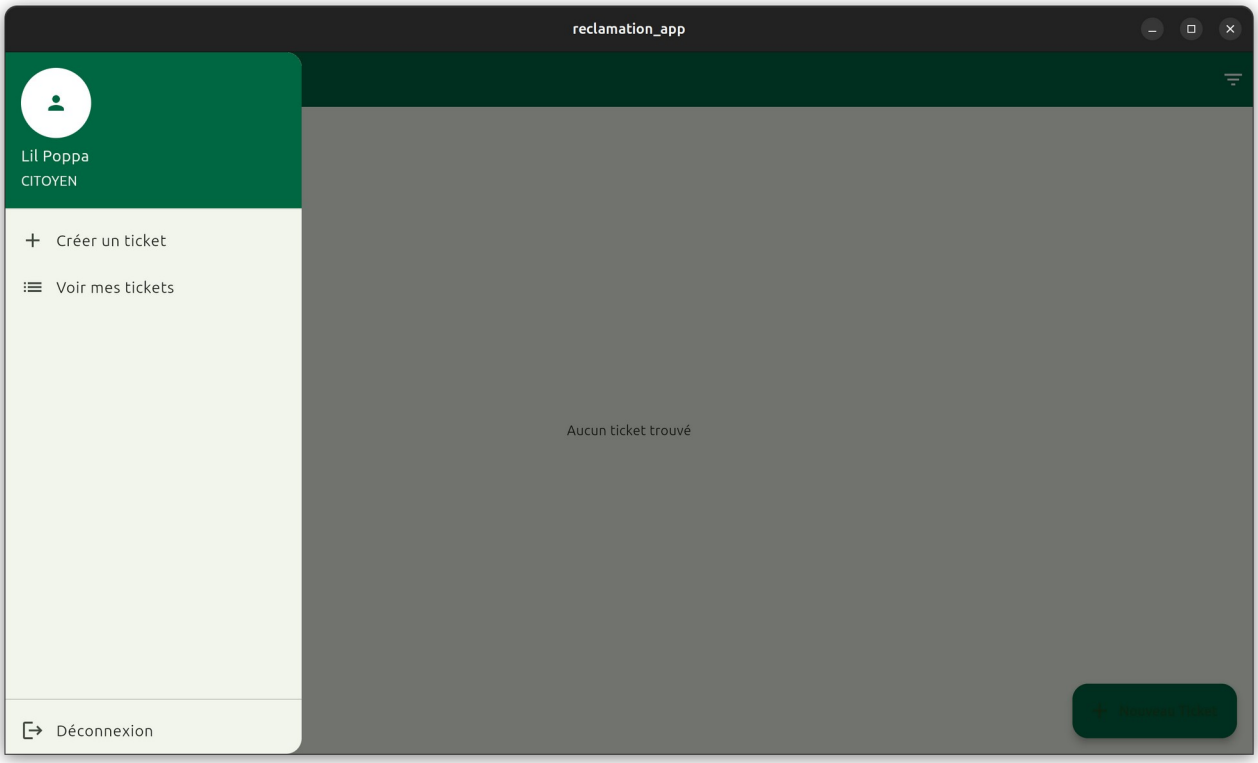


Identifiants

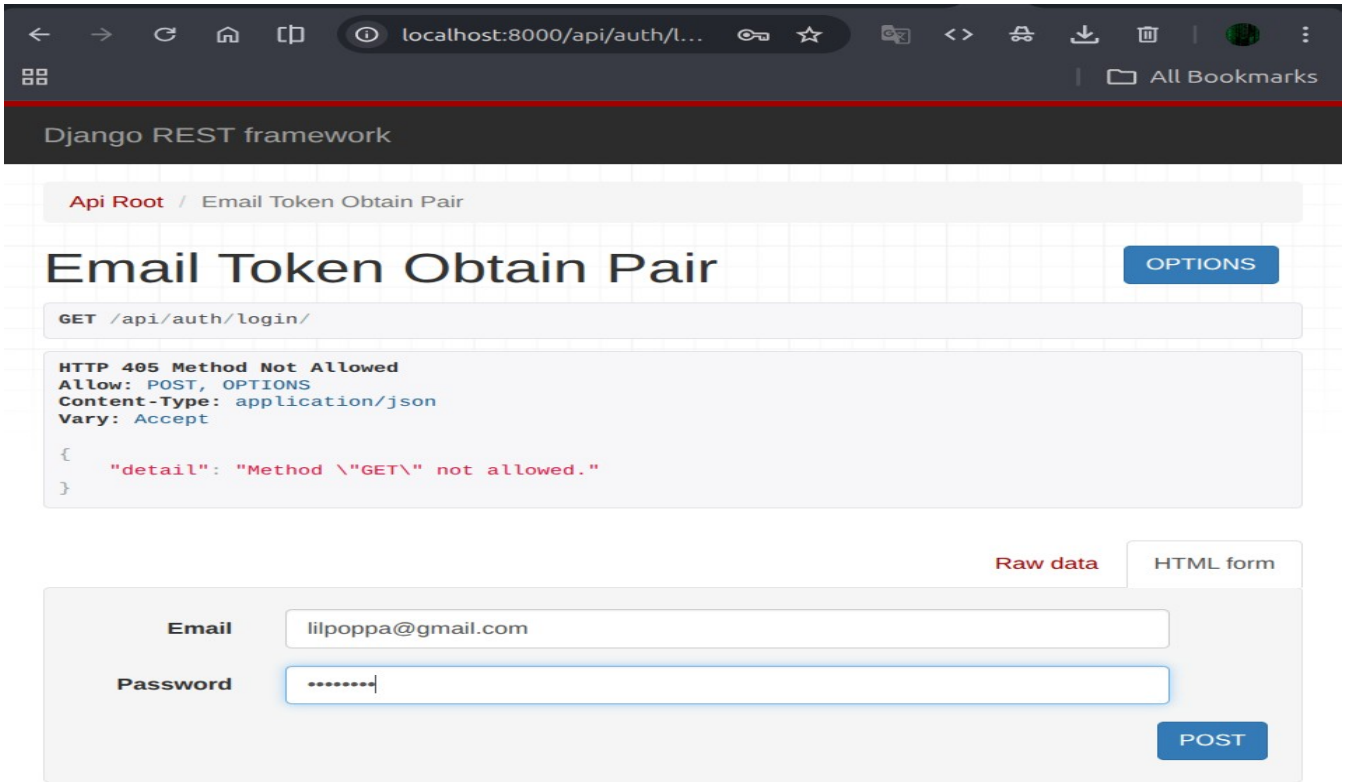
email : *lilpoppa@gmail.com*
password : *pass1234*

Je peux toute fois le faire dans le navigateur en utilisant les l'url ***http://localhost:8000/api/auth/login/*** *puis remplir les même identifiant comme précédament fais avec flutter.*

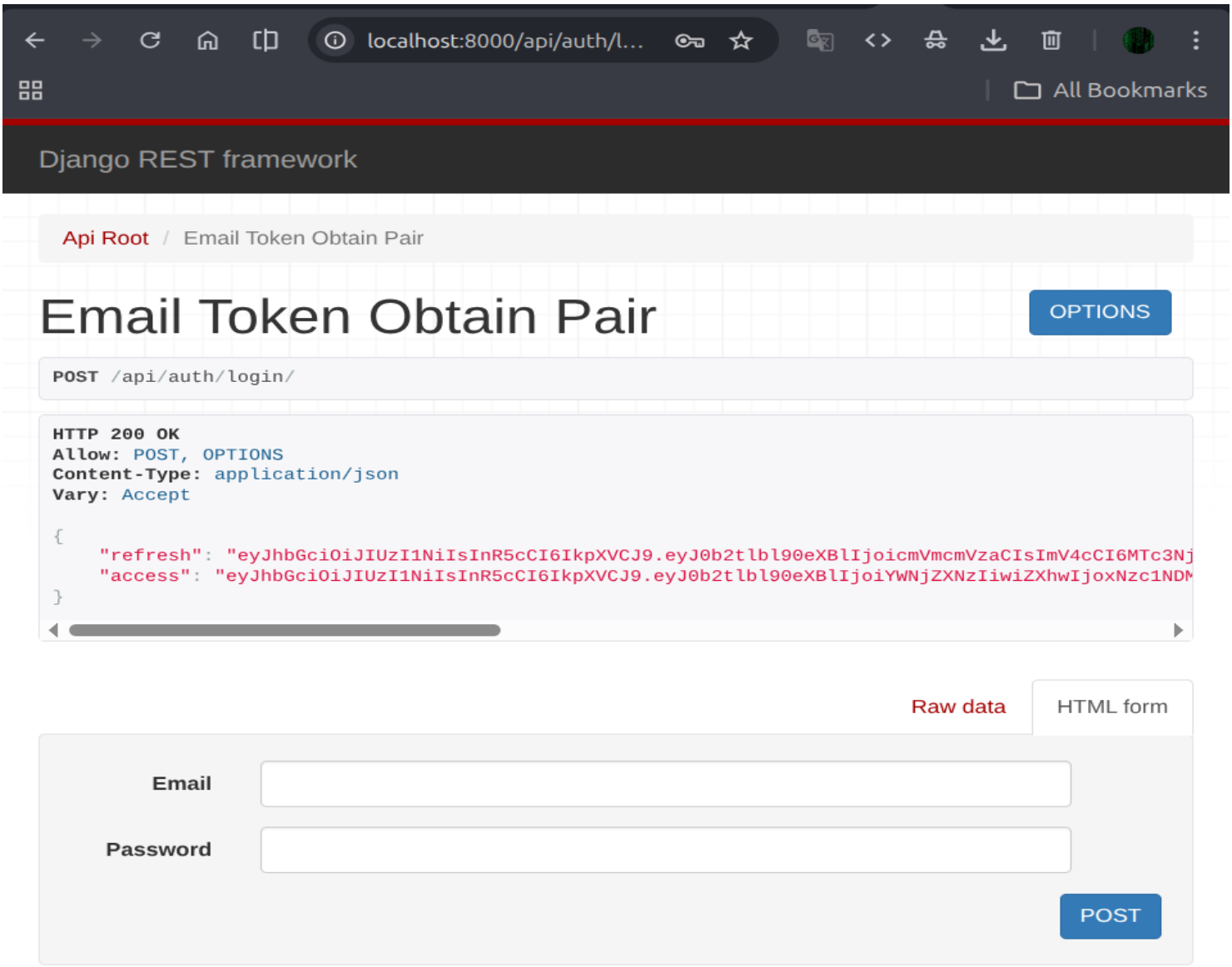
Page d'acceuil aprés connexion.



Page de connexion via l'url



connexion réussi avec l'affichage des Tokens (refresh et access)



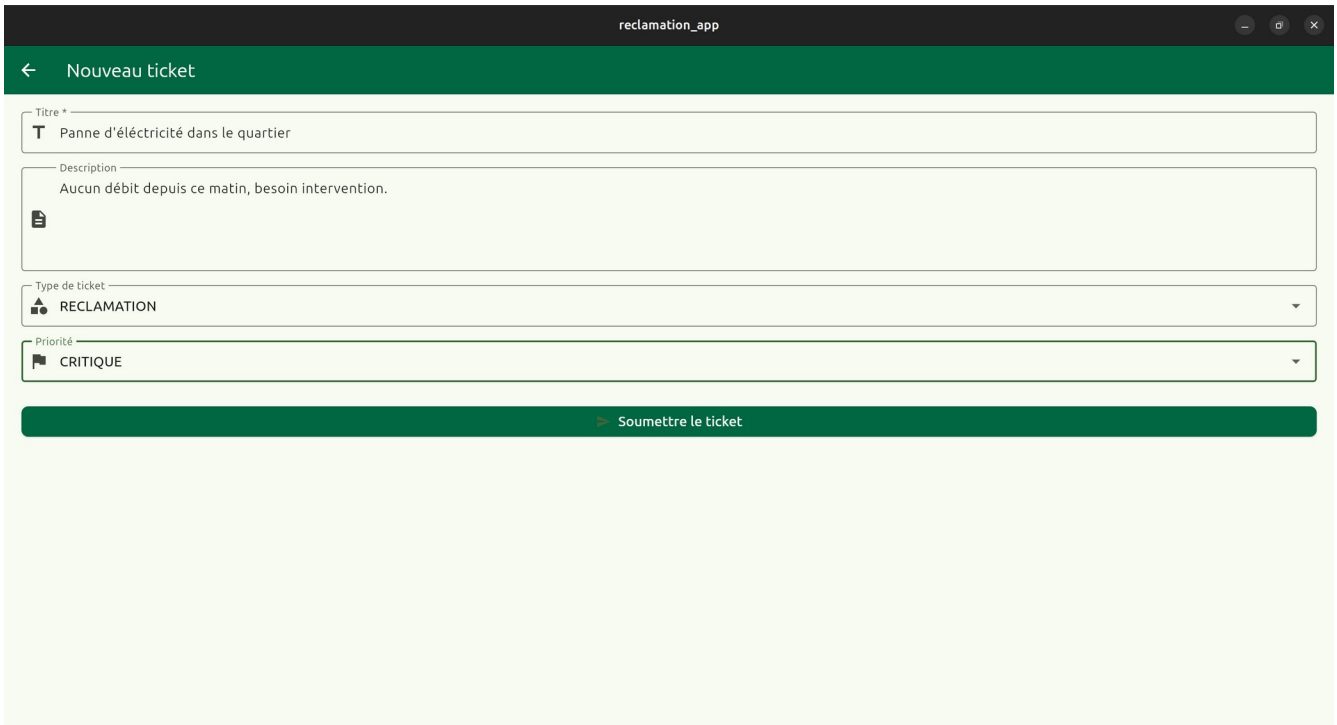
2. Test 2 : Création de Tickets

Sujet :
L'utilisateur remplit le formulaire Flutter et soumet un ticket. Verifier dans l'interface Django Admin que le ticket est bien cree avec le bon auteur.

a. Remplissage du formulaire Flutter et Soumission

Je vais remplire correctement le formulaire de soumission de Ticket, qui souléve le probléme *Panne d'éléctricité dans le quartier.*

Formulaire de création d'un Ticket



b. Connexion en tant qu'admin et vérification

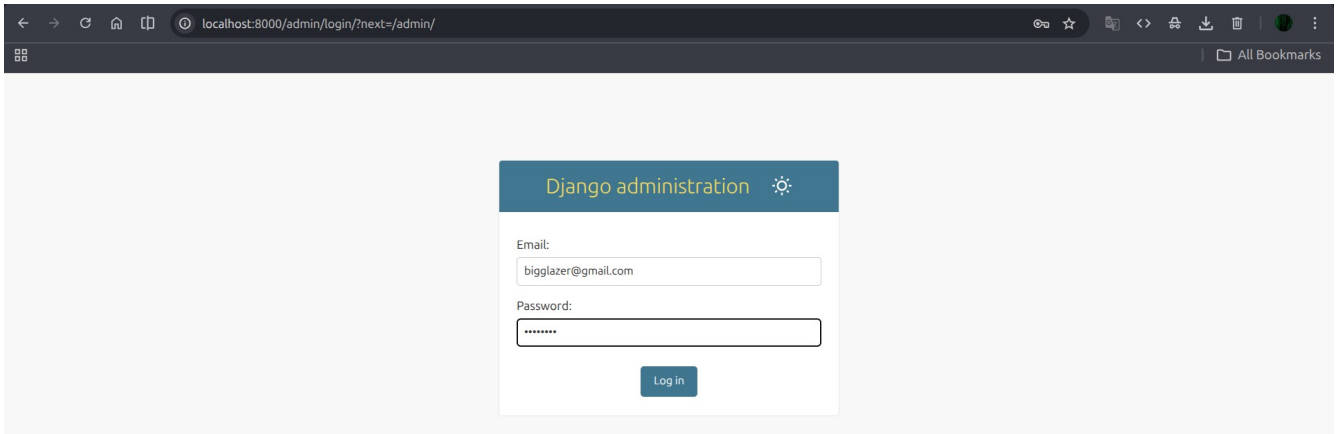
Je vais ouvrir l'url de connexion de l'admin dans mon navigateur Chrome pour m'authentifier.

Email : *bigglazer@gmail.com*

Password : *pass1234*

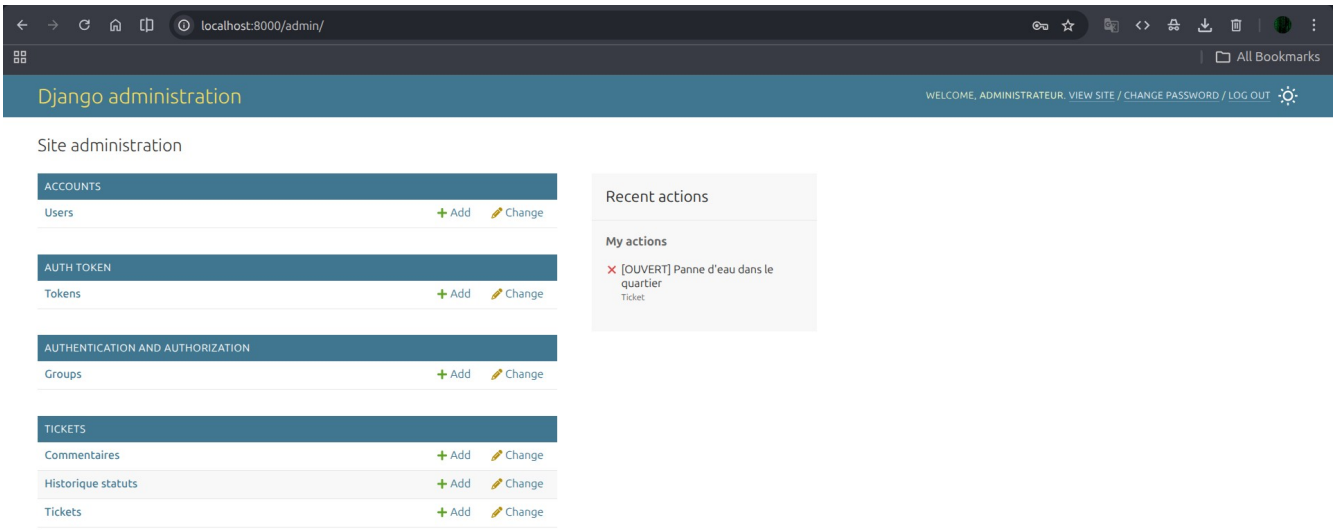
Url : *http://localhost:8000/admin/*

Page de connexion Admin

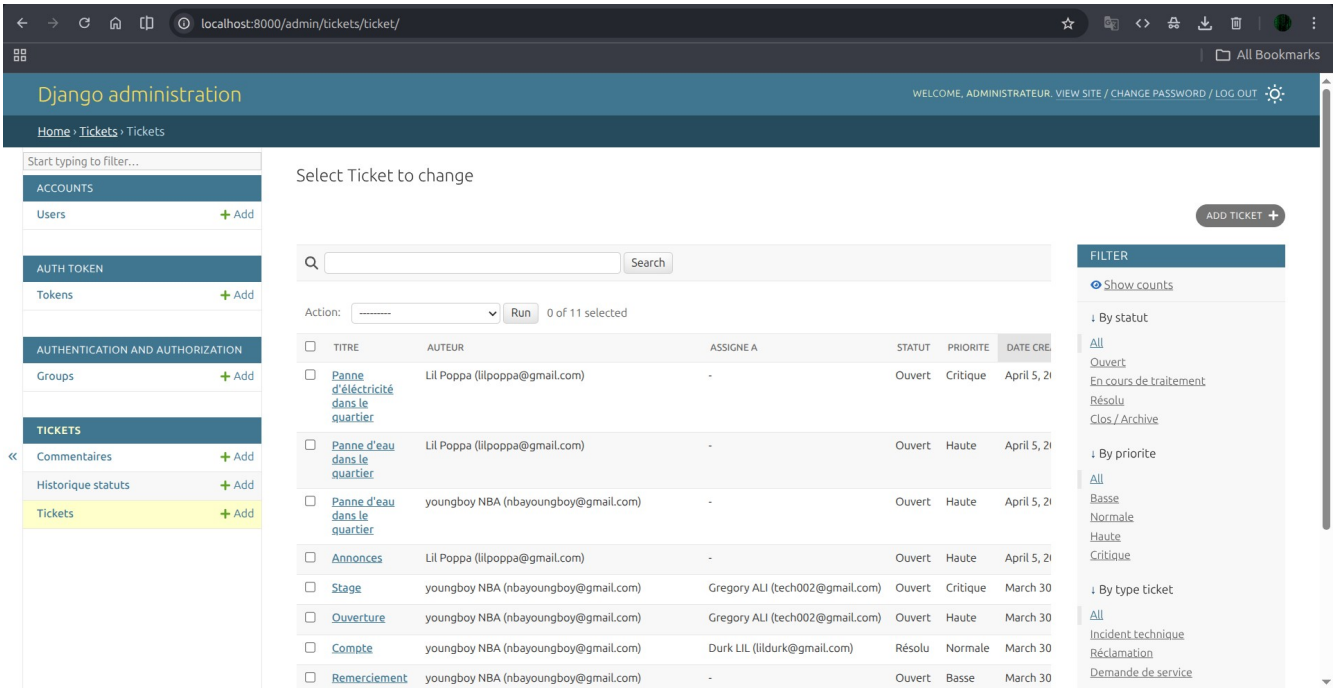


Puis je fais ***Log in***

Page d'acceuil Admin

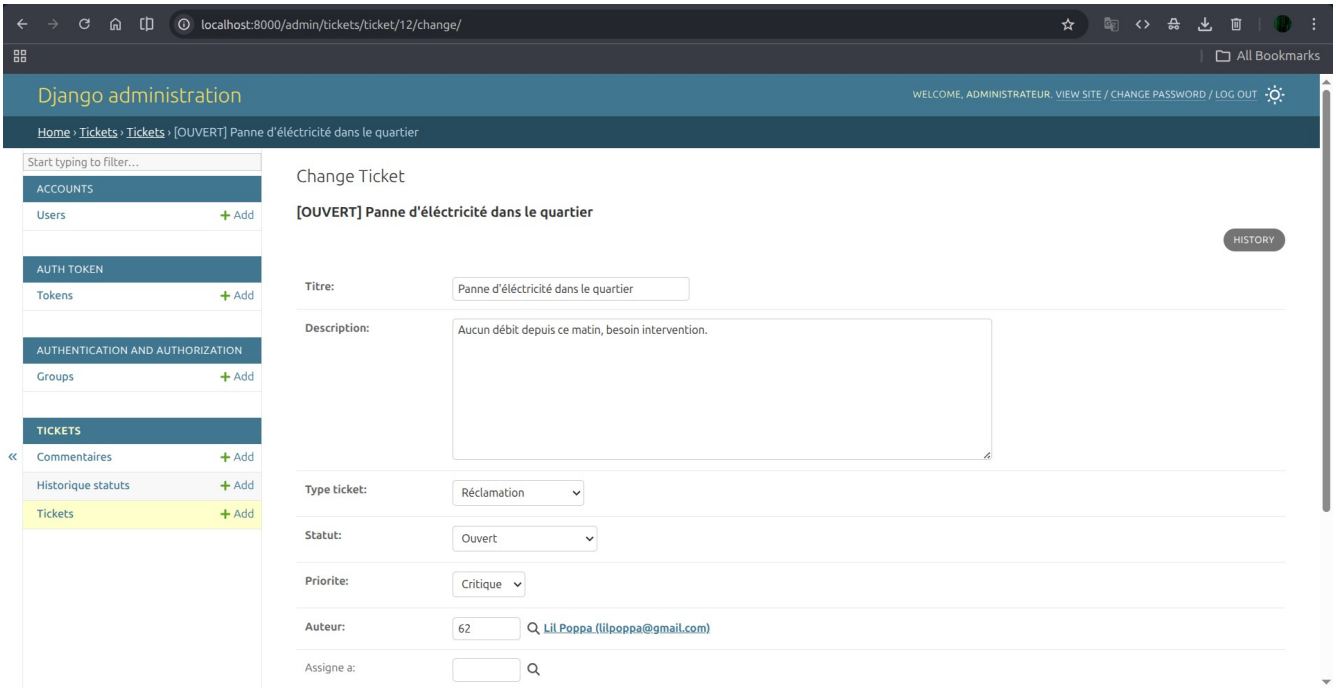


Après je me dirige dans l'onglet Tickets qui se trouve en dessous à gauche



Je peux déjà voir le nouveau Ticket du Nom de ***Panne d'éléctricité dans mon quartier***. Je vais appuyer dessus pour voir les différentes informations y compris le ***nom de l'auteur***.

Information : J'ai plus utiliser les noms des rappeurs US car elles me semblent simple à rétenir (Exemple LIL POPPA).



Nous pouvons voir toutes les informations sur ce ticket avec le nom de l'auteur 'LIL Poppa (lilpoppa@gmail.com)'.

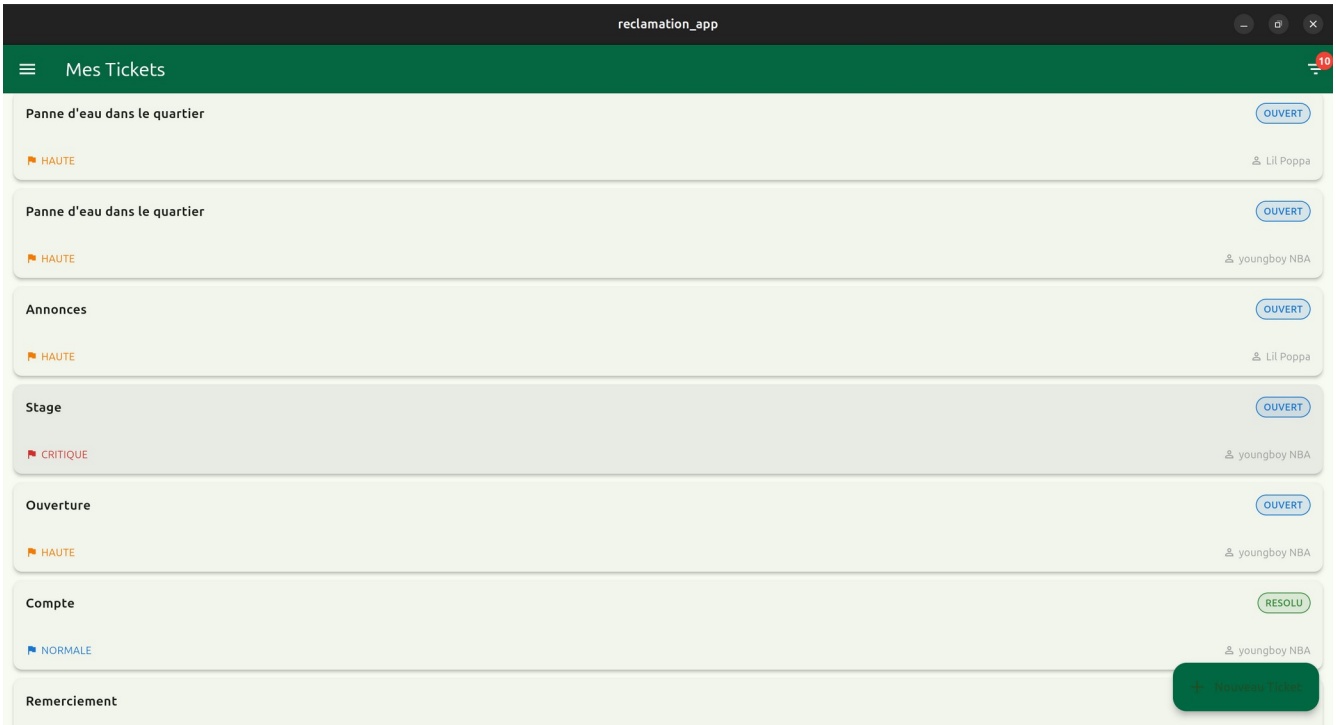
3. Test 3 : Liste des tickets avec Filtre

Sujet :

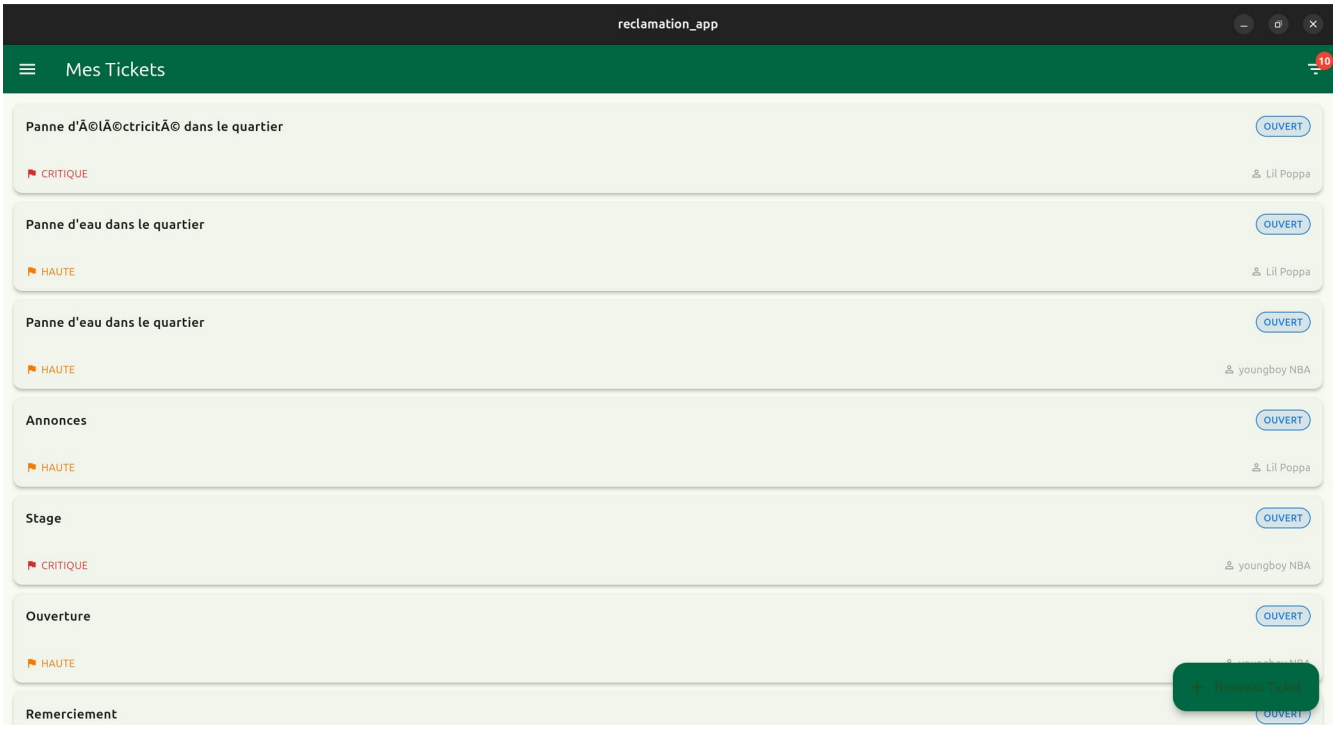
Appliquer un filtre par statut dans Flutter et verifier que les parametres sont bien envoyes dans la query string de l'API.

a. Via Flutter

Voici la liste de tous les tickets via l'interface de l'Admin ***Bigg Lazer***, que je vais essayer de filtrer.

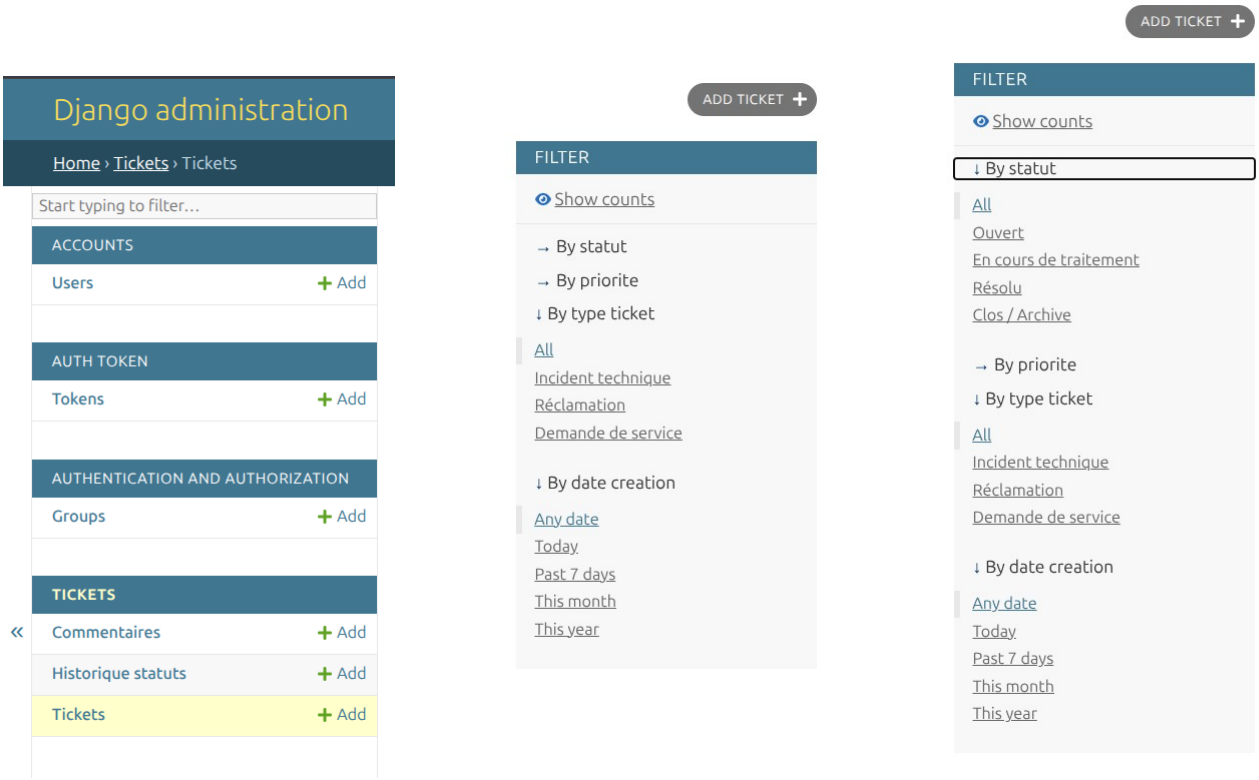


Maintenant je vais filtrer pour avoir la liste des Tickets ***'OUVERT'*** seulement.

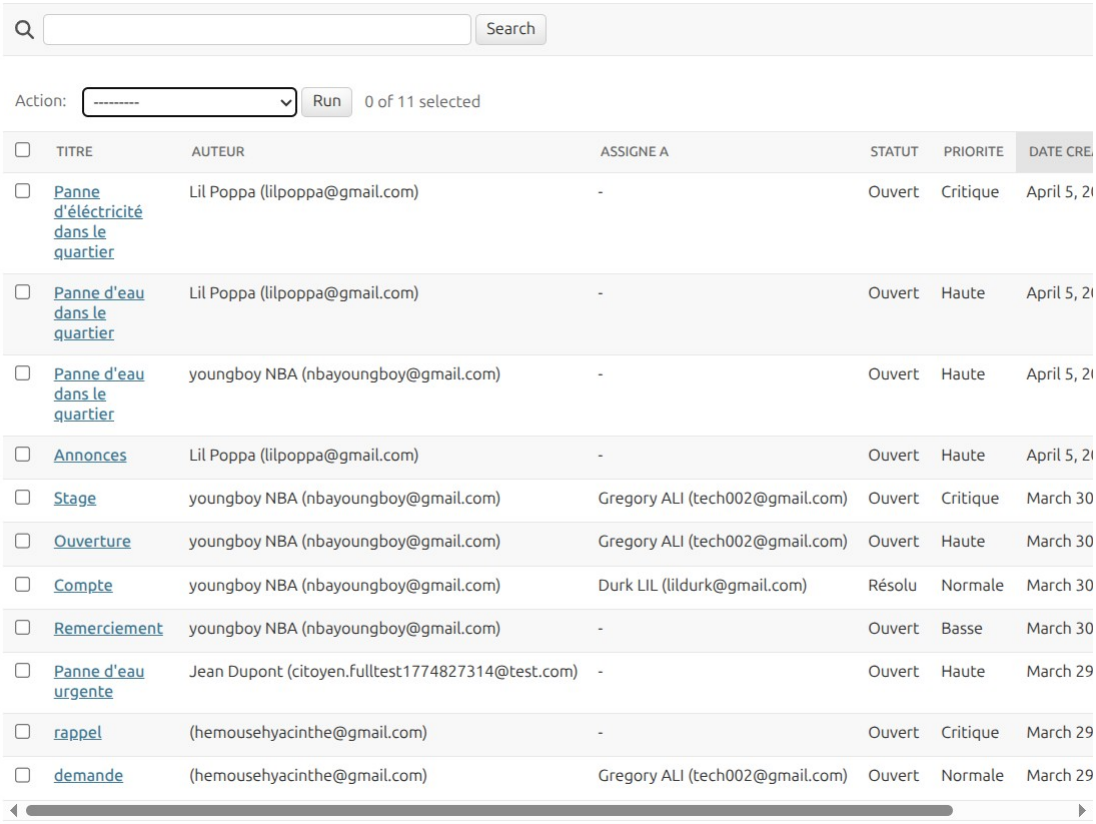


b. Via l'interface Django

Et si je me dirige dans l'onglet ***Tickets> Filter***



Je peux à présent filtrer en ne rendant sur ***'By statut'*** en suite ***'Ouvert'***

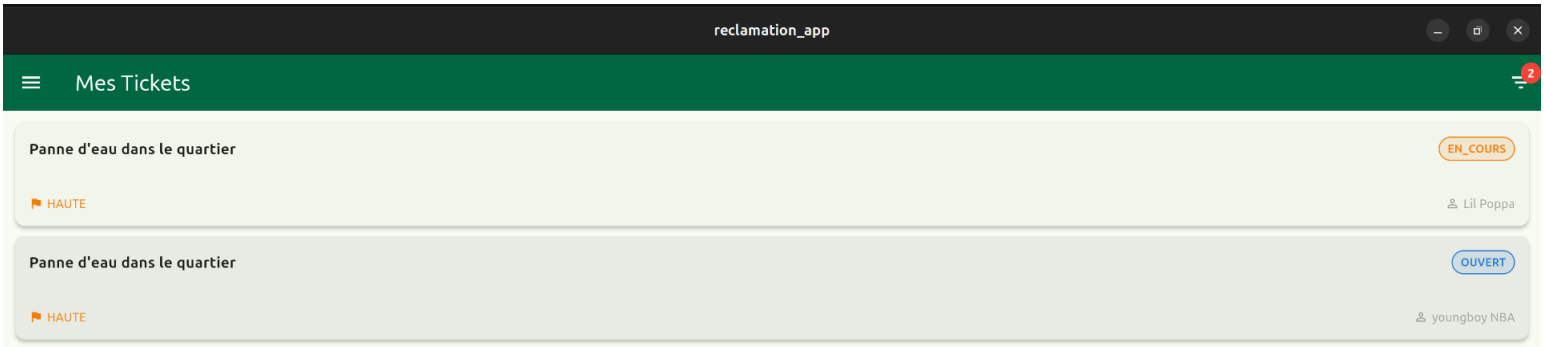


TITRE	AUTEUR	ASSIGNE A	STATUT	PRIORITE	DATE CRE
Panne d'éléctricité dans le quartier	Lil Poppa (lilpoppa@gmail.com)	-	Ouvert	Critique	April 5, 2
Panne d'eau dans le quartier	Lil Poppa (lilpoppa@gmail.com)	-	Ouvert	Haute	April 5, 2
Panne d'eau dans le quartier	youngboy NBA (nbayoungboy@gmail.com)	-	Ouvert	Haute	April 5, 2
Annonces	Lil Poppa (lilpoppa@gmail.com)	-	Ouvert	Haute	April 5, 2
Stage	youngboy NBA (nbayoungboy@gmail.com)	Gregory ALI (tech002@gmail.com)	Ouvert	Critique	March 30
Ouverture	youngboy NBA (nbayoungboy@gmail.com)	Gregory ALI (tech002@gmail.com)	Ouvert	Haute	March 30
Compte	youngboy NBA (nbayoungboy@gmail.com)	Durk LIL (lildurk@gmail.com)	Résolu	Normale	March 30
Remerciement	youngboy NBA (nbayoungboy@gmail.com)	-	Ouvert	Basse	March 30
Panne d'eau urgente	Jean Dupont (citoyen.fulltest1774827314@test.com)	-	Ouvert	Haute	March 29
rappel	(hemousehyacinthe@gmail.com)	-	Ouvert	Critique	March 29
demande	(hemousehyacinthe@gmail.com)	Gregory ALI (tech002@gmail.com)	Ouvert	Normale	March 29

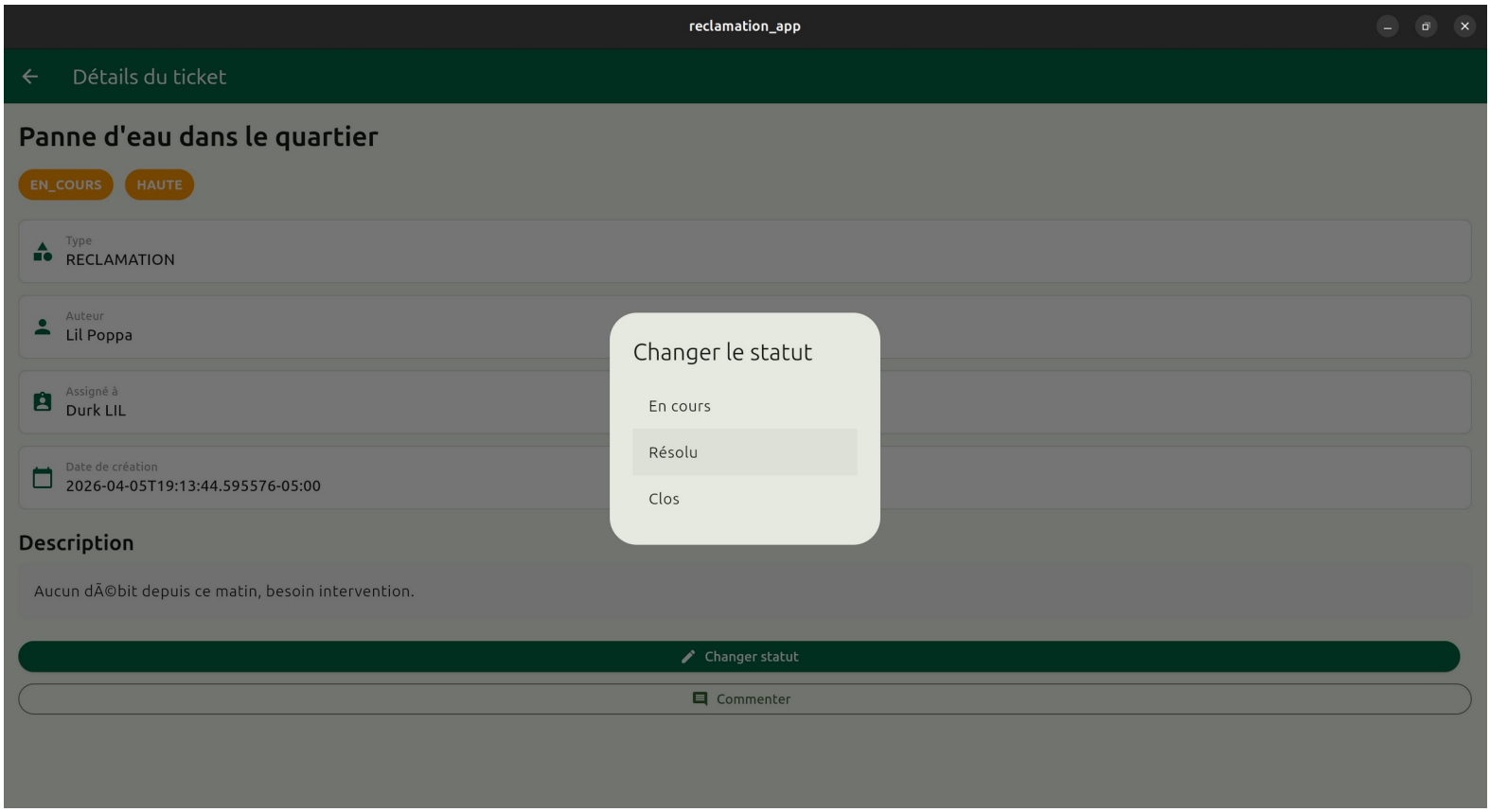
Je vois Exactement la liste que j'avais obtenu avec Flutter.

4. Test 4 : Changement de statut

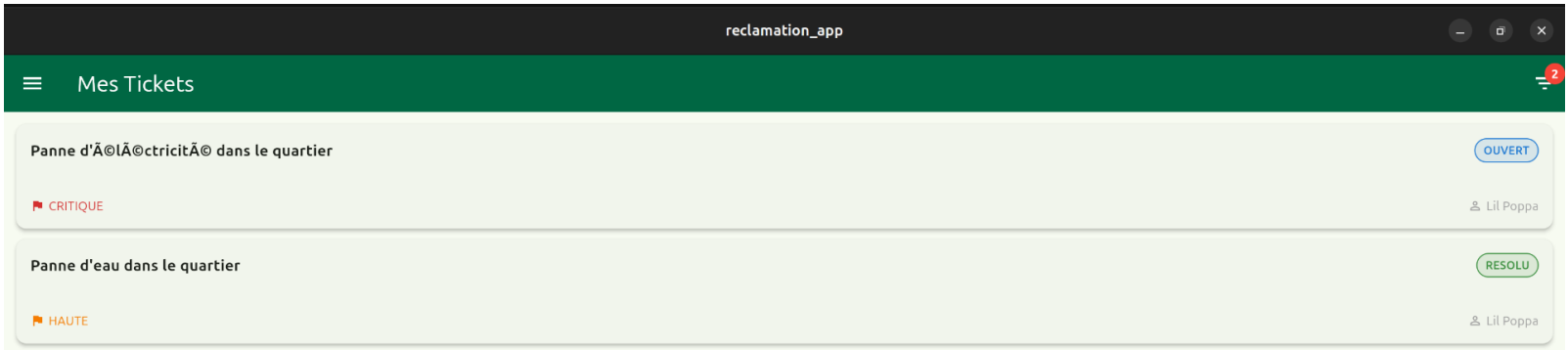
Sujet :
Un technicien change le statut d'un ticket en EN_COURS. Verifier que l'historique est ajoute et que le citoyen voit le nouveau statut.



En tant que Technicien (lildurk@gmail.com) je vais changer le status du Ticket ***Panne d'eau dans le quartier*** de l'auteur lilpoppa@gmail.com qui a un status ***"EN_COURS"*** en ***"RESOLU".***



Je me reconnecte avec les identifiant de 'Lil Poppa' puis observer le changements.



En conclusion, on vois viens le changement s'appliquer aussi chez le Citoyen en question.

5. Test 5 : Changement de statut

Sujet :
Un citoyen essaie d'appeler /changer_statut/. Veri#er que l'API renvoie un code 403 Forbidden.

D'apres mes diagrammes, et l'implementation de mon fichiers *views.py,* la *methode* `@action(detail=True, methods=['patch'], permission_classes=[IsAuthenticated])` *et l'endpoint :* `/api/tickets/<id>/changer_statut/`; la logique importe. Donc si l'utilisateur connecté n'est pas *Admin ou Technicien alors il lui retournera* `return Response({'detail': 'Technicien non assigné au ticket.'}, status=status.HTTP_403_FORBIDDEN)`

```
# ACTION : CHANGER LE STATUT D'UN TICKET
@action(detail=True, methods=['patch'], permission_classes=[IsAuthenticated])
def changer_statut(self, request, pk=None):
    ticket = self.get_object()

    # VÉRIFICATION DES DROITS
    if request.user.role not in (CustomUser.Roles.TECHNICIEN, CustomUser.Roles.ADMIN):
        return Response({'detail': 'Vous n\'avez pas les droits pour changer le statut.'}, status=status.HTTP_403_FORBIDDEN)
    if request.user.role == CustomUser.Roles.TECHNICIEN and ticket.assigne_a != request.user:
        return Response({'detail': 'Technicien non assigné au ticket.'}, status=status.HTTP_403_FORBIDDEN)

    # VALIDATION DU NOUVEAU STATUT
    nouveau_statut = request.data.get('statut')
    if nouveau_statut not in dict(Ticket.Statut.choices):
        return Response({'error': 'Statut invalide'}, status=status.HTTP_400_BAD_REQUEST)
```

En somme un Citoyen reçoit un 403

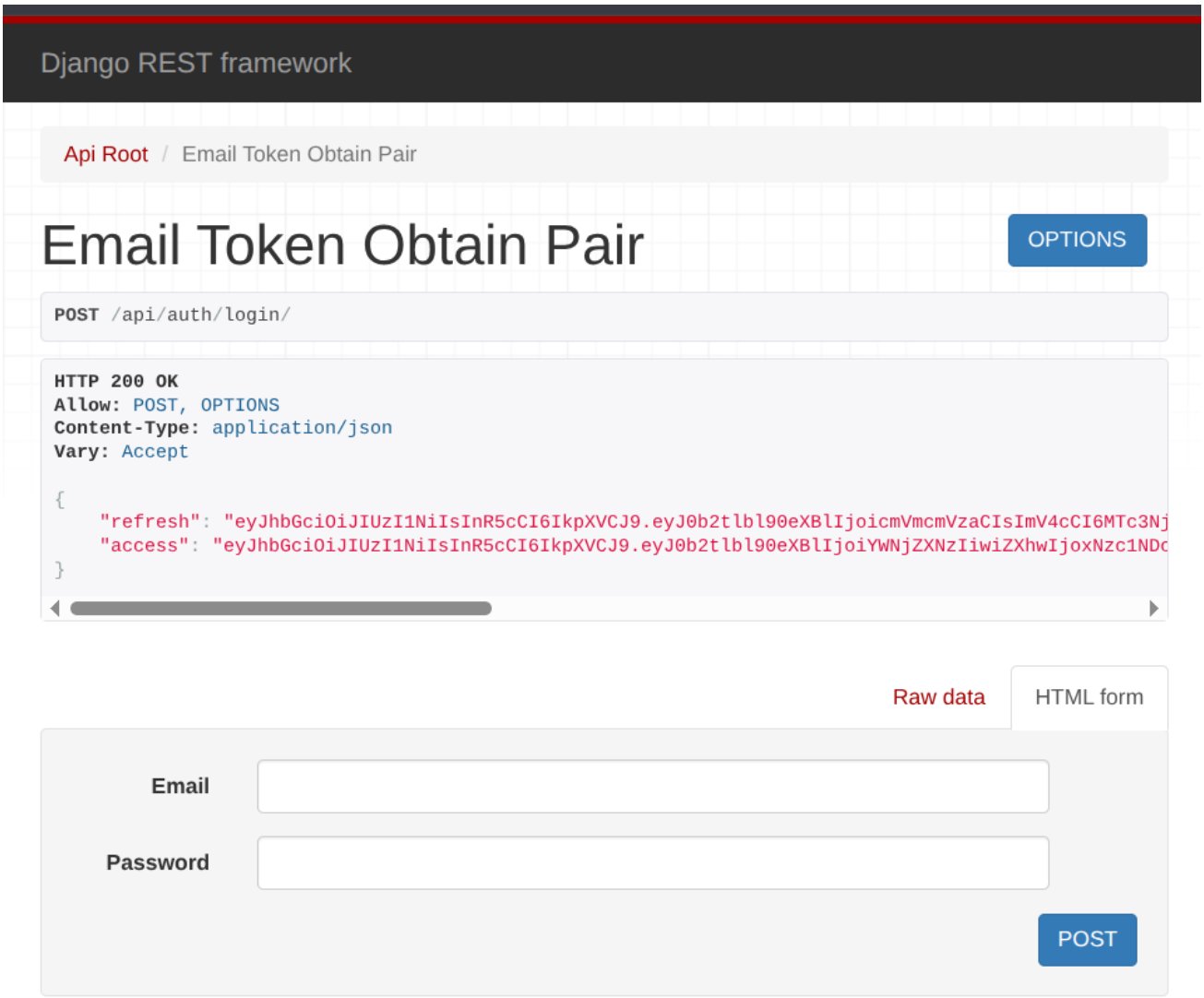
6. Test 6 : Token expire

Sujet :
Attendre l'expiration du token d'acces (modifier ACCESS_TOKEN_LIFETIME a 1 minute pour le test). Verifier que l'application gere la re-connexion ou le refresh automatiquement.

Direction dans mon fichier *settings.py* pour effectuer le changement de durée qui étais *hours=2* à *minutes=1*.

```
# Simple JWT settings
from datetime import timedelta
SIMPLE_JWT = {
    'ACCESS_TOKEN_LIFETIME': timedelta(minutes=1),
    'REFRESH_TOKEN_LIFETIME': timedelta(days=7),
    'AUTH_HEADER_TYPES': ('Bearer',),
}
```

Pour cette partie je vais utiliser ***Postman*** pour faire ces test.



Je me connecte via l'interface Django puis je laisse passer au moins 1 minute et en fin je vais dans Postman pour tester l'expiration comme suit.

Reclamation API / **Token expire**

Save | View Documentation | No Environment

GET http://127.0.0.1:8000/api/tickets/ Send

Params | Authorization | Headers (9) | Body | Scripts | Settings | Code | Cookies

Type: Bearer Token

The authorization header will be automatically generated when you send the request. Learn more about authorization ↗

Token:

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ0b2tlbl90eXBlIjoiYWNjZXNzIiwiZXhwIjoxNzc1NDc4MzI2LCJpYXQiOjE3NzU0NzgyNjYsImp0aSI6Ijc5NzMyNGQxNjdjYjRkMTQ4MjYxNTBiZDdjM2ExMzA3IiwidXNlcl9pZCI6IjYyIn0.oPYiYR0CETm34YbD9glhOUHEcOYRYBBAES9vhhavLG4

Body | Cookies | Headers (11) | Test Results

Status: 401 Unauthorized Time: 7 ms Size: 550 B

Pretty | Raw | Preview | JSON


```
{
    "detail": "Given token not valid for any token type",
    "code": "token_not_valid",
    "messages": [
        {
            "token_class": "AccessToken",
            "token_type": "access",
            "message": "Token is expired"
        }
    ]
}
```


En conclusion apres 1 minutes, j'ai un message qui me dit que le Token est expiré.